

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №2

Выполнил:

Черницын Егор

J3210

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

По выбранному варианту (Система учета личных финансов) привязать ранее созданную HTML/CSS верстку к внешним и внутренним API средствами fetch. Реализовать моковое API (json-server) для управления логикой приложения, авторизации, сохранения данных о счетах и транзакциях пользователей.

- Страницы приложения:
 - Вход (login.html)
 - Регистрация (reg.html)
 - Главная / Дашборд (dash.html)
 - Страница управления счетами (accounts.html)
 - Страница истории транзакций (transact.html)

Ход работы

1. Архитектура и моковое API

В качестве бэкенда для имитации работы базы данных развернут «json-server». Для оптимизации кодовой базы, ускорения загрузки и соблюдения лучших практик скрипты были отделены от HTML-разметки и вынесены в отдельные модульные js файлы.

2. Авторизация и регистрация

Взаимодействие с формами входа и регистрации переведено на асинхронные запросы (fetch). На клиенте реализована валидация данных (проверка длины пароля, проверка на существование пользователя с таким же email в БД). Реализовано сохранение сессии пользователя (ID и имя) в localStorage, а также автоматический редирект неавторизованных пользователей на страницу входа.

3. Управление счетами и транзакциями

Интерфейс дашборда, счетов и транзакций подключен к API. Теперь карточки банковских счетов, баланс и история операций генерируются динамически на основе данных, получаемых из JSON-базы. Реализована отправка POST-запросов из модальных окон для создания новых транзакций и счетов, а также синхронная отправка PATCH-запросов для автоматического пересчета баланса счета с последующим обновлением DOM-дерева.

4. Динамический поиск и фильтрация

На странице истории операций (transact.html) реализована клиентская логика фильтрации. Полученный через API массив транзакций фильтруется с помощью JavaScript по ряду параметров: текстовый поиск (по названию или описанию), категория операции, привязанный счет и диапазон дат (От/До).

5. Визуализация данных и интеграция внешних API

На главной странице (dash.html) реализован сбор статистики. Производится асинхронный запрос к внешнему API (cbr-xml-daily) для получения и вывода актуальных курсов валют (USD, EUR) с индикацией тренда роста/падения.

Для визуализации доходов и расходов используется библиотека Chart.js. Написан алгоритм сортировки данных, который динамически выявляет топ-3 самых объемных категорий пользователя, а остальные расходы/доходы автоматически суммирует и выводит в категорию "Прочее" на диаграмме.

Вывод

В ходе выполнения лабораторной работы мной были освоены принципы клиент-серверного взаимодействия средствами fetch в JavaScript. Получены практические навыки работы с асинхронным кодом, сессиями через localStorage, динамическим рендерингом данных, валидацией форм, а также интеграцией сторонних библиотек и внешних API для получения данных в реальном времени.